

9.3

Преобразование чисел в строки и обратно

00:00–00:15

Интро

Привет! В этом материале мы разберём способы преобразования чисел в строки и обратно на нескольких практических примерах. Это очень часто бывает необходимо в реальных задачах.

00:15–03:19

Практический пример 1. Преобразование чисел в строки

Например, вы читаете файл и получаете данные из этого файла в виде строк, содержащих числа, с которыми вам потом нужно производить какие-то вычисления. В этом случае вам придётся преобразовывать эти строки в числа.

Или наоборот, у вас есть числа, и вам нужно их преобразовать в строки для той же записи в файл или для отправки какого-нибудь отчёта по электронной почте. В этом случае вам придётся преобразовывать числа в строки.

Начнем с преобразования чисел в строки.

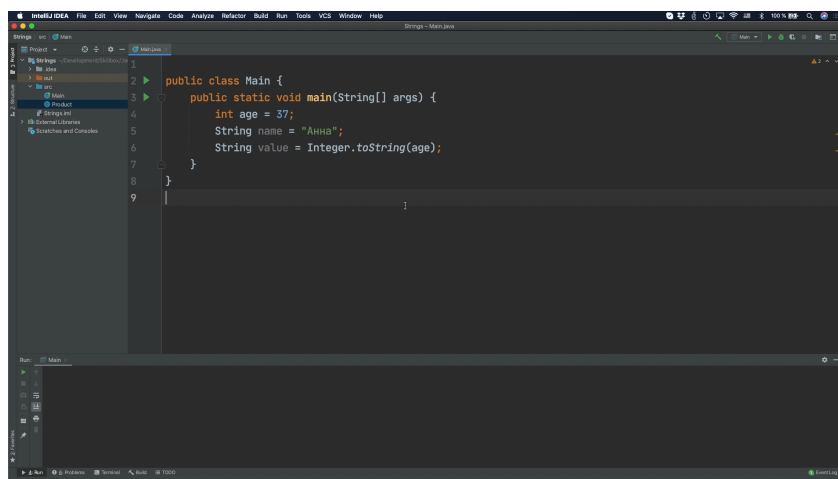
Пусть у нас будет число, возраст некоего человека, и его имя — Анна. Самый простой способ преобразования числа в строку — это автоматическое, неявное преобразование при конкатенации. То есть если мы напишем, допустим:

```
String personInfo = name + " - " + age + "лет";
```

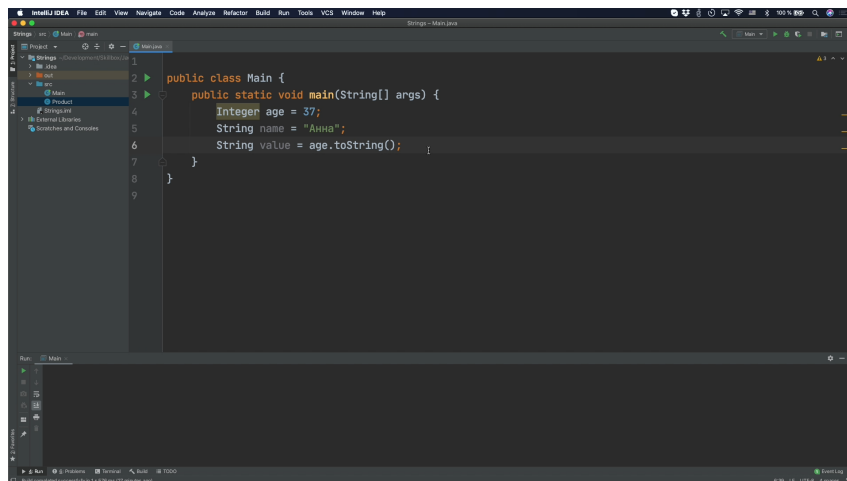
то в этом случае число переменной `age` будет автоматически преобразовано в строку.

Но что делать, если вам необходимо явное преобразование? Например, если вы напишете `string value = age`, то такой код работать не будет, потому что `age` — число, а `value` — строка. Если мы наведём мышку на красное выделение, то увидим сообщение `required type — string, provided type — int`; то есть требуемый тип — строка, предоставленный тип — интеджер.

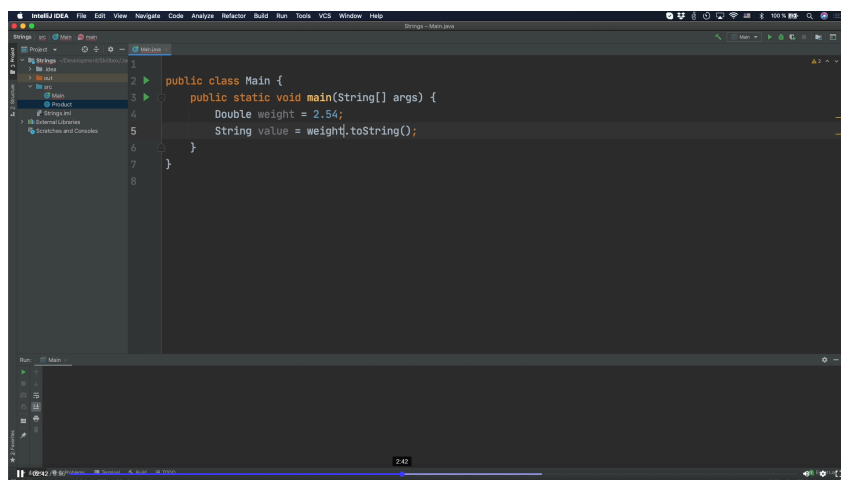
Для того чтобы явным образом преобразовать число в строку, можно воспользоваться статическим методом `toString` соответствующего класса — в данном случае у нас будет соответствовать числу `age`. И если в этот метод передать переменную `age` в качестве параметра, то эта переменная будет преобразована в строку.



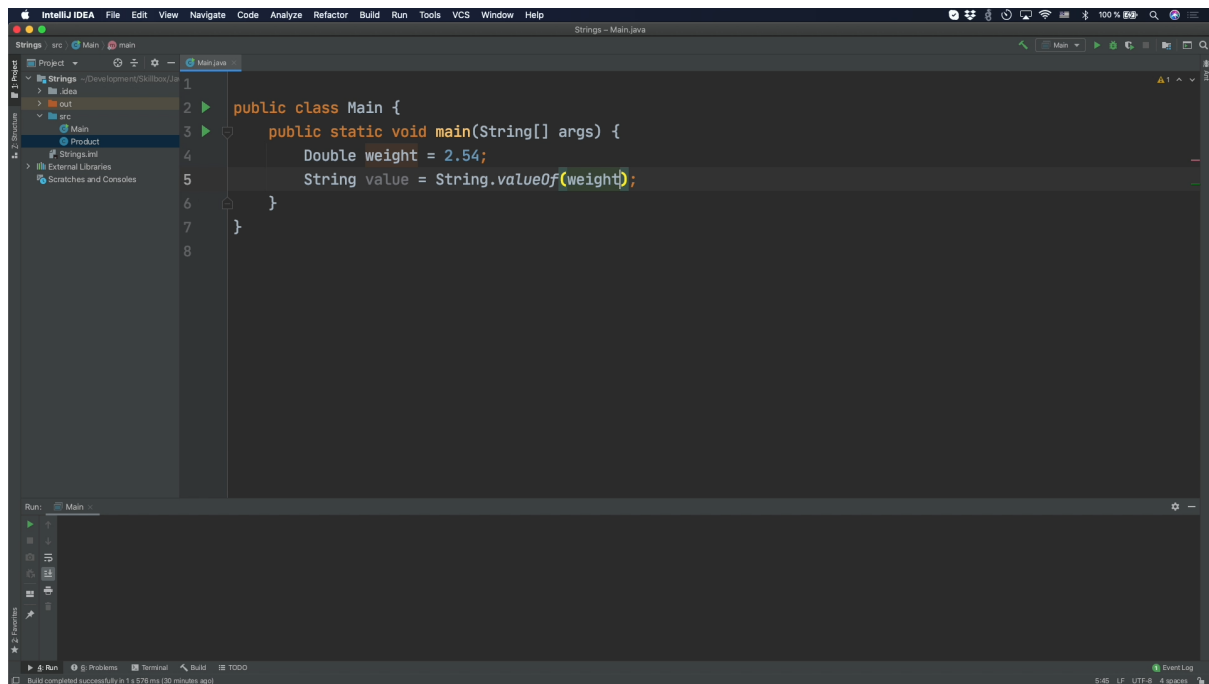
Также если ваше число было объектом соответствующего класса, а не примитивом, у этого объекта тоже можно вызвать метод `toString`.



Метод toString есть у всех классов чисел. Вот, например, у double:



Для преобразования чисел строки можно также использовать статический метод valueOf класса string — он существует для всех примитивов и даже для объектов, поэтому в данном случае мы можем передать ему переменную weight, которая является объектом.



Таким образом, есть четыре способа преобразования чисел в строки:

- Автоматическое неявное при конкатенации.
- С использованием статических и нестатических методов соответствующих классов чисел.
- А также с использованием статического метода value of класса string:

Число → строка

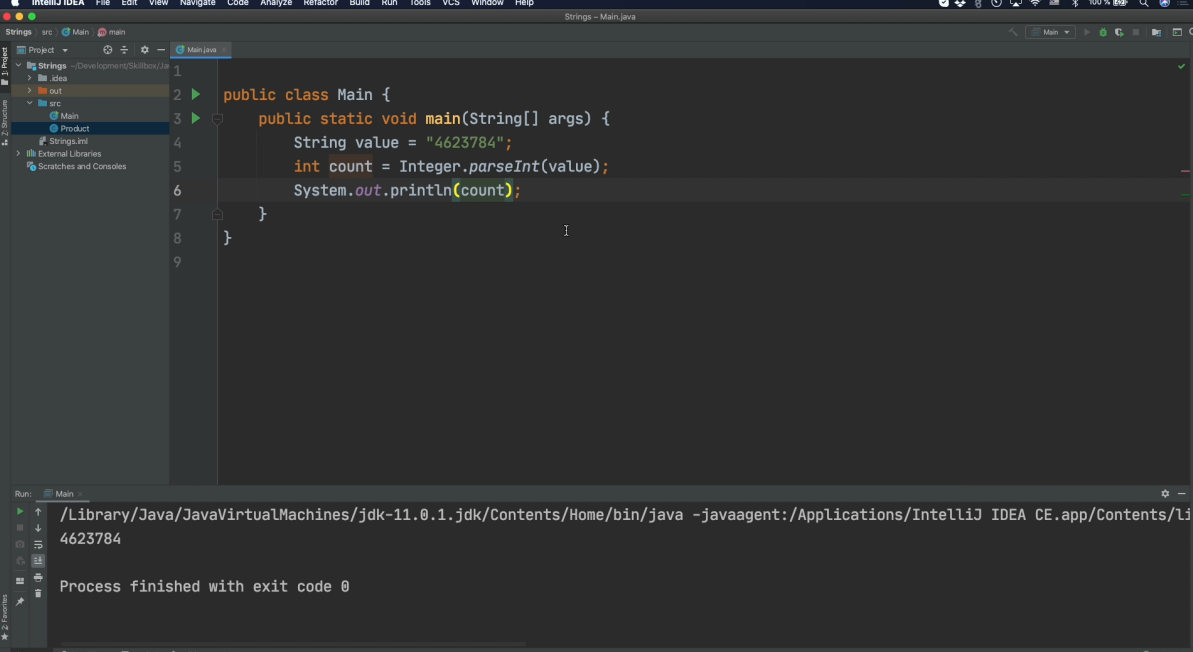
- 1 Неявное при конкатенации
- 2 Double.toString()
- 3 Double value = 2.56;
value.toString();
- 4 String.valueOf(value);

03:19–05:29

Преобразование строки в число

Теперь давайте посмотрим, как преобразовать строку в число. Сделаем это на практическом примере.

Пусть есть некая строка, содержащая число, и основной способ преобразования — это методы парсинга строк соответствующих классов чисел. Давайте создадим число и воспользуемся методом `parseInt` класса `Integer`, а после проверим результат — всё работает верно:



The screenshot shows the IntelliJ IDEA IDE with a Java project named 'Strings'. The 'Main.java' file is open, showing the following code:

```
1 public class Main {  
2     public static void main(String[] args) {  
3         String value = "4623784";  
4         int count = Integer.parseInt(value);  
5         System.out.println(count);  
6     }  
7 }  
8  
9
```

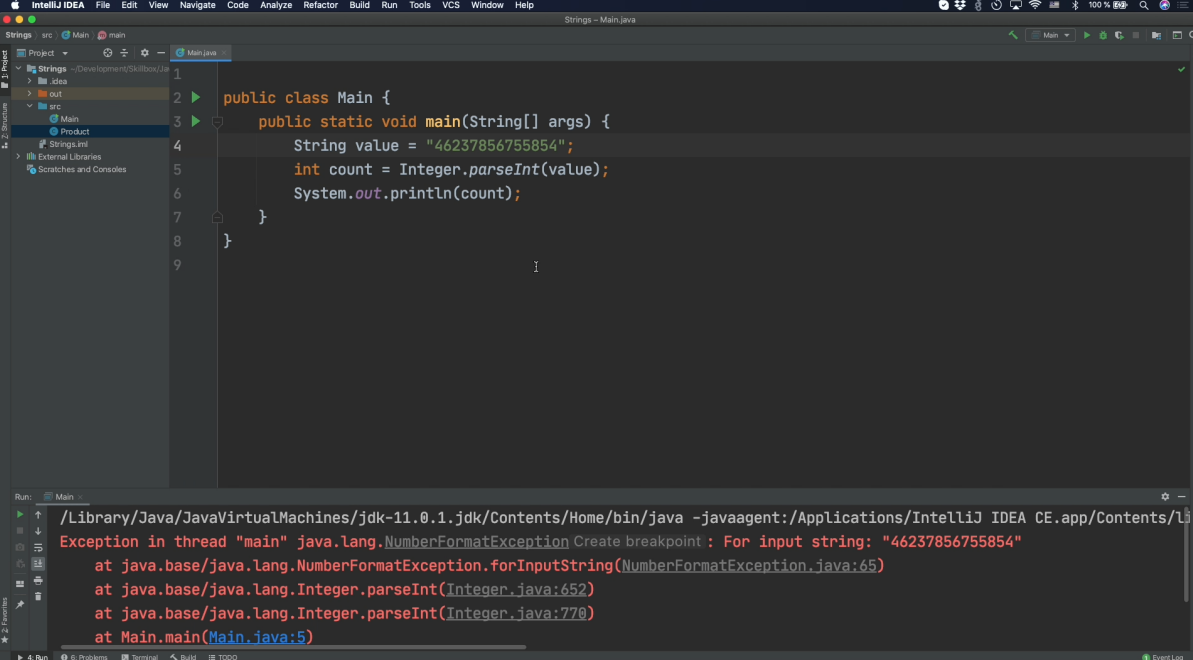
The 'Run' tab at the bottom shows the execution output:

```
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/lib  
4623784  
Process finished with exit code 0
```

The status bar at the bottom indicates 'Build completed successfully in 1 s 327 ms (previous: 490 ms)'.

Если число, которое мы парсим, будет превышать допустимые пределы, то произойдёт исключение — здесь мы видим, что

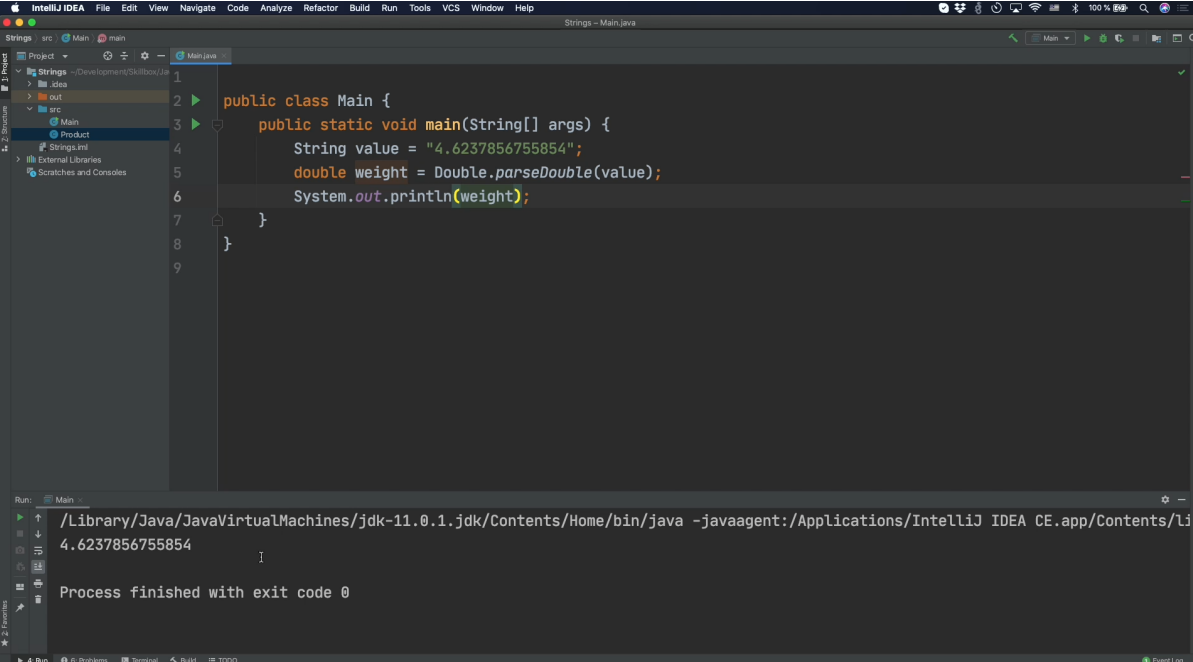
произошло исключение — NumberFormatException.



The screenshot shows the IntelliJ IDEA IDE with a Java file named `Main.java`. The code defines a `Main` class with a `main` method that takes a `String[] args` parameter. Inside the `main` method, a `String value` is assigned the value `"46237856755854"`, and then `Integer.parseInt(value)` is called to parse it into an `int`. The result is printed using `System.out.println(count)`. The IDE's Run window at the bottom displays the following exception stack trace:

```
Exception in thread "main" java.lang.NumberFormatException: Create breakpoint : For input string: "46237856755854"
    at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:65)
    at java.base/java.lang.Integer.parseInt(Integer.java:652)
    at java.base/java.lang.Integer.parseInt(Integer.java:770)
    at Main.main(Main.java:5)
```

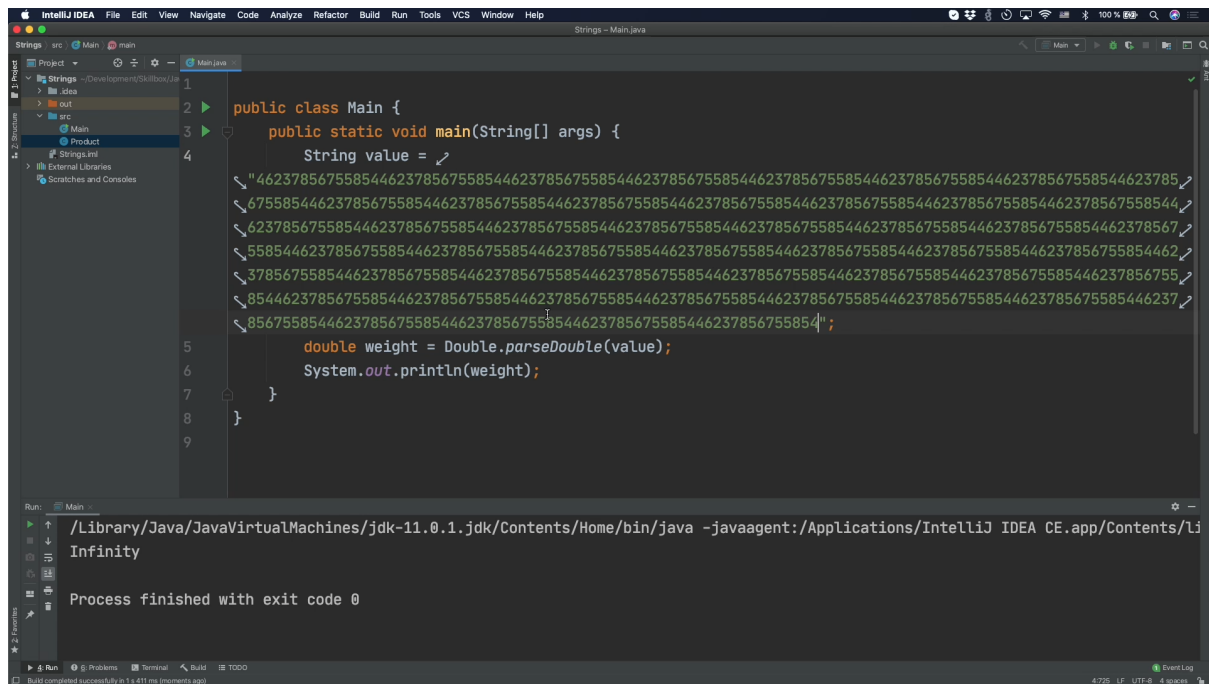
У классов чисел с плавающей точкой тоже есть такие методы парсинга. Давайте посмотрим на примере:



The screenshot shows the IntelliJ IDEA IDE with a Java file named `Main.java`. The code defines a `Main` class with a `main` method that takes a `String[] args` parameter. Inside the `main` method, a `String value` is assigned the value `"4.6237856755854"`, and then `Double.parseDouble(value)` is called to parse it into a `double`. The result is printed using `System.out.println(weight)`. The IDE's Run window at the bottom displays the following output:

```
4.6237856755854
Process finished with exit code 0
```

В случае если число слишком маленькое или слишком большое, метод `parseDouble` выдаст специальное значение `infinity`, то есть бесконечность:



Либо значение $(-)$ бесконечность, если это число отрицательное.

Бесконечность и минус бесконечность — это специальные значения чисел с плавающей точкой float и double.

Таким образом, преобразовывать строки в числа можно с использованием статических методов соответствующих классов. Эти методы есть у всех классов чисел. При этом важно помнить, что числа в строках, которые мы парсим, должны соответствовать классам, которые мы используем для такого парсинга, иначе может произойти исключение.

Строка → число

1 Методы парсинга у классов-чисел:

Integer.parseInt()
Double.parseDouble()

Числа в строках должны соответствовать классам, методы которых мы используем для парсинга, иначе может произойти `NumberFormatException`.

05:29–5:56

Итоги

Итак, мы разобрали различные способы преобразования чисел в строки и обратно, а также познакомились со специальными методами и их особенностями. Вы уже знаете, что строки состоят из символов, если только это не пустые строки, и из следующего материала вы детально узнаете, как работать со строками с точки зрения символов и отдельно с символами строк.